

ТЕМА 3. МОДЕЛ НА ДАННИТЕ

ДАННИ. КОМПЮТЪРЕН МОДЕЛ НА ДАННИТЕ

- **Данни** – това е общо понятие за всичко това, с което оперират компютърните системи
- **Модел на данните** – при изучаването и моделиране на данните в една програма т.нар. компютърен модел могат да се разграничат три аспекта (характеристики, модела)*:
 - абстрактен модел на данните - **същност**
 - знаков модел на данните - **синтаксис**
 - физически модел на данните – **на ниво хардуер**

* Тази теоретична постановка не е общоприета. Тук се използва само, за да се разграничат свойствата и физическото представяне от конкретната реализация в езика C++

АБСТРАКТЕН МОДЕЛ НА ДАННИТЕ - СЪЩНОСТ

Абстрактният модел (АМ) на данните определя присъщите им свойства, като игнорира детайлите на тяхното физическо представяне

Абстракцията на данни е разделянето на логическите свойства на данните от тяхното конкретно приложение

- Средствата за абстрахиране с цел описание на данните в ЕП се развиват по естествен начин в съответствие с нуждите от такива средства:
 - Формални математически модели (математически структури): реална алгебрична система, вектори, матрици, множества, съжително и предикатно смятане и т.н.
 - Абстрактни модели данни, които са били създадени или уточнени специално за нуждите на информатиката: масив, запис, файл, стек и т.н.
- **Видове АМ: прости (базови абстракции на данните) и съставни (структурни абстракции на данните)** според своята структурност

ЗНАКОВ МОДЕЛ НА ДАННИТЕ - СИНТАКСИС. ТИП НА ДАННИТЕ В ЕП

Знаков модел (ЗМ) – моделът на данните на ниво език за програмиране, т.е. знаково описание на съответно избраните абстрактни модели, основава се на концепцията за *тип на данните*

- **Тип данни (*data type*) – същност:**
 - основна концепция, заложена в представянето на данните в ЕП
 - основна семантична компонента на ЕП, която обхваща същността(характерът) на данните
 - осигурява представяне на данните по начин възможно по-естествен за човека
 - улеснява обработката на данните
 - напр. типа на реалните числа (реален тип данни)
- **Тип данни – определение:** Всеки *тип данни* е съвкупност от три множества:
 - **множество от стойности**, които представляват константите на типа, т.е. набор от стойности, които могат да се присвояват на променливи от този тип
 - **множество от операции**, които могат да се извършват над неговите елементи (за реалния тип данни това са събиране, изваждане, умножение и т.н.)
 - **множество от релации** валидни над множеството от стойности на типа (за реалния тип данни това са еднаквост/равно, наредба и т.н.)

Освен това в ЕП за някои типове се поддържа и множество от т.нар. стандартни функции и процедури. Това е ключова характеристика на ЕП от високо ниво.

СИСТЕМА ОТ ТИПОВЕ ДАННИ В ЕП. СИЛНО И СЛАБО ТИПИЗИРАНЕ

Система от типове данни на езиките за програмиране е съвкупността от правила използвани за изграждане и организиране на съвкупността от типове

- **Изисквания:**

- всеки тип данни трябва да бъде предварително дефиниран
- всеки елемент данни принадлежи на точно един от съществуващите в езика типове данни(има се предвид един конкретен момент)
- на всеки елемент на данните трябва да бъде присвоен тип
- всеки оператор или операция в езика изисква операнди от определени типове и дава резултат от определен тип

- В зависимост от строгостта на спазване на тези правила ЕП се класифицират от **силно типизирани (strongly typed)** - Pascal, Java C# към **слабо типизирани (weakly typed)** - LISP, JavaScript, Perl

- **Предимства:**

- надеждност – може да се провери правилната употреба на типа
- улеснена четливост
- преносимост и изменяемост – ако представянето се промени, не се налага променяне на приложните програми

ФИЗИЧЕСКИ МОДЕЛ НА ДАННИТЕ – НИВО ХАРДУЕР

Физическият модел (ФМ) на данните е начинът, по който стойностите на даден тип данни се съхраняват в паметта на компютъра – място и правила за съхранение, количество заемана памет

- **Същност:**

- Правилното съхраняване на стойностите от различен тип в повечето случаи е **грижа на компилатора** на ЕП
- Пример: С++ разполага стойностите на променливите, декларирани в главната функция на дадена програма (автоматични променливи) в т.нар. стек (*stack, stack frame*). Компилаторът резервира съответното количество памет за различните величини (константи и променливи) при срещане на техните декларации

- **Ограничения:**

- Тъй като всеки "жив" ЕП е реализиран на компютър (физическо устройство), то това неизбежно означава, че физическият модел съответства на знаковия и абстрактния модел, но при известни ограничения
- Пример: Множествата на реалните и целите числа са безкрайни, но се реализират като крайни в определен диапазон, тъй като компютърната памет не е безкрайна

ИЗУЧАВАНЕ ИЛЮСТРИРАНО С ПРИМЕРИ НА C++

- Абстрактен модел – определение, приложение и реализация в ЕП
- Физически модел в езика C++
- Знаков модел на данните в езика C++ :
 - Синтаксис за деклариране на типа и на величини от този тип
 - Множество от стойности на типа
 - Операции
 - Релации
 - Стандартни функции

БАЗОВИ АБСТРАКЦИИ НА ДАННИТЕ

Най-често са абстрактни модели, които се използват масово и в други области на човешкото знание, като например алгебра на реалните числа, алгебра на целите числа и т.н.

В ЕП те се реализират чрез *прости* (примитивни) *типове данни*

ПРОСТИ ТИПОВЕ ДАННИ В ЕП

- **Определение:** *Простите* типове (*simple data types*) са такива, чийто стойности са неделими (атомарни)
- **Видове според начина им на дефиниране:**
 - обикновено са реализирани като **стандартни** (вградени, предварително-дефинирани) **типове** в ЕП – целочислени, реални, символни, логически и т.н.
 - **дефинирани от потребителя** (ЕП предлагат средства, чрез които потребителят може сам да дефинира свои собствени типове)
- **Константите** от прост тип – могат да се задават директно по начина близък до общоприетия:

102 (целочислена)	-2.16 или 0.01E-3 (реални)
true и false (логически)	'a' 'z' '%' 'я' (символни)
- **Видове според природата им – посочващи и скалярни**

Посочващите типове се отличават от всички останали по това, че стойностите им са адреси от паметта на компютъра

① Тук ще бъдат разгледани само скалярните прости типове

ПРОСТИ СКАЛАРНИ ТИПОВЕ ДАННИ В ЕП

- **Определение** – единична числова стойност
- Скаларът (на латински: *scalaris*, от *scala* – стълба) е величина, чиято стойност може да се изрази с едно число.
- **скаларните** (*scalar*) типове имат две основни свойства:
 - стойностите им са крайно множество и представляват **един единствен компонент т.е. може да представят една стойност**
 - между стойностите им е определена строга наредба (за тях са определени всички операции за сравнение $<$, $>$, $=$, и т.н.)
- **Приложение:** Когато програмата трябва да обработва прости данни, между които има наредба те могат да се моделират с някой скаларен тип данни. Например парична сума, мярка, пореден номер, елемент от азбука и т.н.

СИСТЕМА ОТ ТИПОВЕ В C++

- **Характеристики:**

- Естествената *система от типове* на C++ е сходна с тези на някои други езици за програмиране

- **Правила:**

- Всеки тип данни трябва да бъде деклариран, за да може да се използва в програмата – *предварително деклариран* (стандартен) или *декларирани от потребителя* по специални синтактични правила
- *Простите типове* (с малки изключения) в езика са предварително декларирани, като са им съпоставени стандартни имена: `int`, `float`, `double`
- *Синонимни имена* (синоними) на типовете могат да се декларират явно чрез ключовата дума `typedef` (това не е декларация на нов тип):
 - **синтаксис** – не може да се декларират в рамките на дефиниция на функция:
`typedef <декларация на тип> <име-синоним>;`
 - **пример:**
`typedef unsigned long ulong; //СИНОНИМ ulong`

СИСТЕМА ОТ ТИПОВЕ НА C++

FUNDAMENTAL DATA TYPES

Group	Type names*	Notes on size / precision
Character types	char	Exactly one byte in size. At least 8 bits.
	char16_t	Not smaller than char. At least 16 bits.
	char32_t	Not smaller than char16_t. At least 32 bits.
	wchar_t	Can represent the largest supported character set.
Integer types (signed)	signed char	Same size as char. At least 8 bits.
	<i>signed short int</i>	Not smaller than char. At least 16 bits.
	<i>signed int</i>	Not smaller than short. At least 16 bits.
	<i>signed long int</i>	Not smaller than int. At least 32 bits.
	<i>signed long long int</i>	Not smaller than long. At least 64 bits.
Integer types (unsigned)	unsigned char	(same size as their signed counterparts)
	unsigned short int	
	unsigned int	
	unsigned long int	
	unsigned long long int	
Floating-point types	float	
	double	Precision not less than float
	long double	Precision not less than double
Boolean type	bool	
Void type	void	no storage
Null pointer	decltype(nullptr)	

ПРОМЕНЛИВИ

- Променливата е място за съхраняване на информация, която може да се променя или не по време на изпълнение на програмата. На това място се задава име - за достъп до информацията.

Тя осигурява възможност за:

- запазване на информация;
 - извличане/четене на запазената информация;
 - промяна на запазената информация.
- Характера на информацията, която ще съхраняваме в една променлива определя т.нар. тип. Например, ако искаме да съхраним и обработим информация за цената на една стока и използваме променлива, то тя трябва да бъде от числов тип.

ДЕКЛАРИРАНЕ НА ПРОМЕНЛИВИ В C++

Типът на всяка променлива, която се използва в програмата се декларира задължително, чрез **синтаксиса**:

<декларация на тип> <списък имена на променливи>;

Където *<декларация на тип>* е:

- указване името на типа

```
int Myint;
```

```
ulong ul; // Еквивалентно на "unsigned long ul;"
```

- директна декларация на типа:

```
struct mystruct
```

```
{ int i; float f; char c;} var_ms;
```

```
typedef struct mystruct
```

```
{ int i; float f; char c;} mystruct;
```

```
mystruct ms; // Еквивалентно на "struct mystruct ms;"
```

ДЕФИНИРАНЕ НА ПРОМЕНЛИВИ В C++

Дефиницията на променлива включва освен елементите от декларацията на една променлива и указание каква памет да бъде заделена за тази променлива, както и присвояване на начална стойност на променливата (инициализация)

- **Синтаксис:**

*<декларация на тип> <име на променлива> = <инициализиращ израз>
незад , ...;*

- **Декларация на типа :**

- всеки валиден тип от езика;
- `auto` или `decltype` (от версия C++11) – “автоматично” определяне на типа на променливата, но е инициализирана.

- **Примери:** `int x = 10; auto mark = 6; int x (0);
int x {0}; // (от версия C++11) и decltype(x) b; // int b;`

- **Правила:**

- при задаване на инициализиращият израз се спазват различни ограничения в зависимост от типа памет, който използва променливата
- в някои случаи, ако променливата не е инициализирана, то тя се инициализира автоматично обикновено със стойност 0 (отнася се за глобални и статични, за локалните се „хвърля грешка“ - VS2017)

ИМЕНУВАНЕ НА ПРОМЕНЛИВИ – ПРЕПОРЪКИ

- ✓ Имената трябва да са описателни и да подсказват за какво служи дадената променлива. Например за име на човек подходящо име е **personName**
- ✓ Трябва да се използват само латински букви.
- ✓ В много езици е прието променливите да започват с малка буква и да съдържат малки букви, като всяка следваща дума в тях започва с главна буква. Например препоръчително име е **firstName**, а не **firstname**
- ✓ Името на променливите трябва да не е нито много дълго, нито много късо – само трябва да е ясно за какво служи променливата в контекста, в който се използва
- ✓ Да се внимава за главни и малки букви, тъй като се прави разлика между тях. Например **ace** и **Ace** са две различни променливи

ДЕФИНИРАНЕ НА КОНСТАНТНИ СТОЙНОСТИ В C++

Типът на константите може да бъде:

- Литерали - определен от компилатора автоматично

```
cout << "Angel : "; // константа от тип низ
```

```
cout << 180.0; // реална константа
```

- явно деклариран:

- **синтаксис:**

```
const <тип> <име> = <израз>;
```

- **пример:**

```
const double Neutral_pH = 7.0; /* pH на водата */
```

```
const int CREDITMARK = 65;
```

```
const char QUERY = '?';
```

```
const double PI = 3.1415926535898;
```

```
const double SOMENUM = 1.235E75;
```

```
const double DEGREES_TO_RADIAN = PI / 180.0;
```

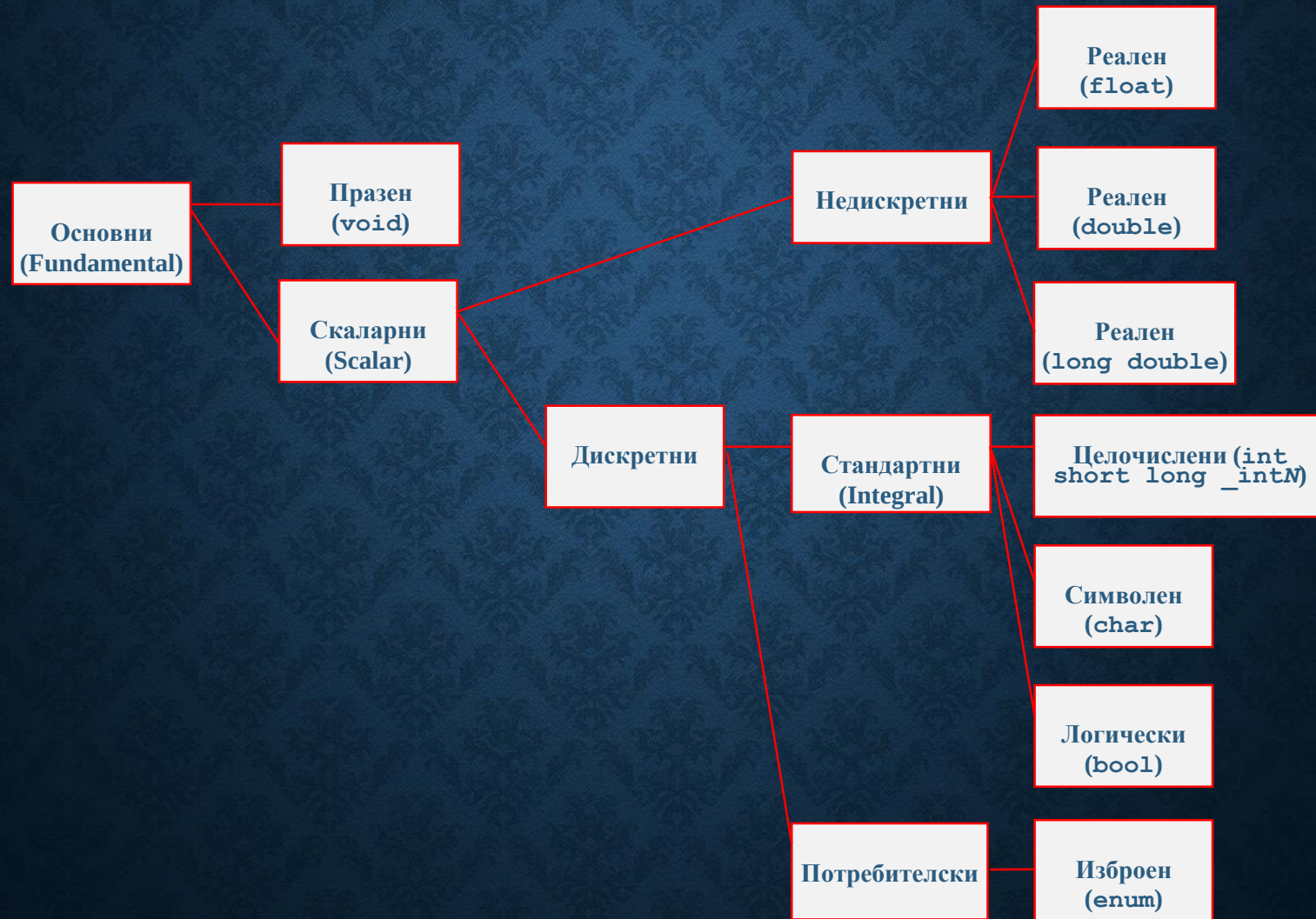
- **предимства:**

- повишава четливостта на програмата

- улеснява евентуални модификации на програмата

- За дефиниране на константи може да се използва и директивата на препроцесора `#define` (за деклариране на макроси): `#define g 32.0`

ПРОСТИ ТИПОВЕ ДАННИ В C++: КЛАСИФИКАЦИЯ



ПРОСТИ ТИПОВЕ ДАННИ В C++ – ЗНАКОВ МОДЕЛ

Ако игнорираме указателите, съществуват само **три основни категории**:

- **целочислени типове** (*integral*) – предназначени за обработка на цели числа (положителни и отрицателни в десетична, също и в осмична и шестнайсетична бройна система – започват съответно с 0 и 0x)
- **типове с плаваща запетая** (*floating*) – предназначени за обработка на числа, които могат да имат дробна част
- **празен, недействителен тип** (`void`) – описва празно множество от стойности



Не могат да се декларираат променливи от тип `void` (използва се за деклариране на резултата на функции, които не връщат стойност или за деклариране на указатели от произволен или неизвестен тип)



В рамките на първите две категории различните типове се различават само по обхвата на числата, които те представят, което зависи от големината на представянето (броя на използваните байтове/битове) и дали представянето е явно или неявно (`char`, `bool`, `enum`)

ЦЕЛОЧИСЛЕНИ ТИПОВЕ В C++ – ЗНАКОВ МОДЕЛ

Тип	Кратко	Физическо представяне
int		Цяло число от -2,147,483,648 до 2,147,483,647
short int	short	Цяло число в диапазона -32768 to +32768
long int	long	Цяло число от -2,147,483,648 до 2,147,483,647
long long int (C++11)	long long	Цяло число от -9,223,372,036,854,775,807 до 9,223,372,036,854,775,807
__intn		Цяло число с фиксиран размер (в битове). n може да бъде 8, 16, 32 или 64
char		Символ – буква, цифра или друг символ от дефинираното множество от символи (в Microsoft C++, е Unicode (UTF-8 или 16))
bool		Логическите стойности true и false

- Към декларацията може да се добави ключовите думи signed/unsigned – със/без знак (с изключение на тип bool)
- Примери:

```
short aCount; // екв. на short int aCount; signed short aCount
char aGenderFlag; // M = male, F = Female
long FailCount, ECount, DCount, CCount, BCount, ACount;
bool Condition=true;
```

ЦЕЛОЧИСЛЕНИ ТИПОВЕ В C++ – МНОЖЕСТВО ОТ СТОЙНОСТИ И ФИЗИЧЕСКИ МОДЕЛ

- Множество от стойности – цели числа в определен интервал, могат да се задават в различна бройна система

- `int d = 42;`
- `int o = 052;`
- `int X = 0X2A;`
- `int b = 0b101010; // C++14`

- Физическо представяне

Тип	Размер
char, unsigned char, signed char	1 байт
short, unsigned short	2 байта
int, unsigned int	4 байта
long, unsigned long	4 байта
long long	6 байта
bool	1 байт

ТИПОВЕ С ПЛАВАЩА ЗАПЕТАЯ В C++ – ЗНАКОВ МОДЕЛ

- АМ на реалния тип не се отличава от математическото понятие за реална алгебрична система. Този тип не е дискретен, т.е. както е известно от математиката множеството от стойностите му не е изброимо, тъй като между всеки две реални числа a и b съществуват безброй много други реални числа c , за които $a < c < b$.
- Деклариране и обхват:

Т и п	Ф и з и ч е с к о п р е д с т а в я н е
<code>float</code>	Реално число (ниска точност)
<code>double</code>	Реално число (стандартна точност)
<code>long double</code>	Реално число (стандартна точност)

- Примери:
`float money;`
`double pi;`

ТИПОВЕ С ПЛАВАЩА ЗАПЕТАЯ В C++ – МНОЖЕСТВО ОТ СТОЙНОСТИ И ФИЗИЧЕСКИ МОДЕЛ

- Множество от стойности – числа с плаваща запетая с единична(7 цифри след запетаята), двойна(15 циф.) или „разширена“ точност- според стандарта IEEE 754
- Физическо представяне:

Т и п	Р а з м е р
<code>float</code>	4 bytes
<code>double</code>	8 bytes
<code>long double</code>	8 bytes

integral	16	signed	$\pm 3.27 \cdot 10^4$	-32767 to 32767
		unsigned	0 to $6.55 \cdot 10^4$	0 to 65535
	32	signed	$\pm 2.14 \cdot 10^9$	-2,147,483,647 to 2,147,483,647
		unsigned	0 to $4.29 \cdot 10^9$	0 to 4,294,967,295
	64	signed	$\pm 9.22 \cdot 10^{18}$	-9,223,372,036,854,775,807 to 9,223,372,036,854,775,807
		unsigned	0 to $1.84 \cdot 10^{19}$	0 to 18,446,744,073,709,551,615
floating point	32	IEEE-754	$\pm 3.4 \cdot 10^{\pm 38}$ (~7 digits)	$\pm 3.402,823,4 \cdot 10^{38}$
	64	IEEE-754	$\pm 1.7 \cdot 10^{\pm 308}$ (~15 digits)	$\pm 1.797,693,134,862,315,7 \cdot 10^{308}$

ПРОСТИ СТАНДАРТНИ ТИПОВЕ В C++ – МНОЖЕСТВО ОТ ОПЕРАЦИИ И РЕЛАЦИИ

Като следствие от свойствата на скаларните типове, в езика C++ за всички прости стандартни типове могат да се прилагат следните релации и операции:

- ♦ всички сравнения `==`, `!=`, `<`, `>`, `<=`, `>=`

- ♦ **пре/пост-операция за нарастване** (*pre/post increment operators*) – добавя 1 към аргумента си

```
int count=5;  
++count; /* PRE Increment, добавя 1 към count */  
count++; /* POST Increment, добавя 1 към count */
```

- ♦ **пре/пост-операция за намаляване** (*pre/post decrement operators*) – изважда 1 от аргумента си (освен за тип `bool`)

```
int count=5;  
--count; /* PRE Increment, изважда 1 от count */  
count--; /* POST Increment, изважда 1 от count */
```

- ♦ **логически операции:**

`&&` (логическо "и"), `||` (логическо "или"), `!` (логическо "не")

- ♦ **извеждане** (с `cout`) и **въвеждане** (с `cin`), като за стойности от тип `bool` се използва техният целочислен еквивалент

ПРОСТИ СТАНДАРТНИ ТИПОВЕ В C++ – АРИТМЕТИЧНИ ОПЕРАЦИИ

Операция	Знак	Пример	Стойност на sum/money преди	Стойност на sum/money след
Умножение	*	sum = sum * 2;	4	8
Деление	/	sum = sum / 2;	4	2
Деление	/	money = money / 2;	4.2	2.10000
Събиране	+	sum = sum + 2;	4	6
Изваждане	-	sum = sum - 2;	4	2
Нарастване prefix	++	++sum;	4	5
Намаляване prefix	--	--sum;	4	3
Нарастване postfix	++	sum++;	4	5
Намаляване postfix	--	sum--;	4	3
Модул	%	sum = sum % 3;	4	1

 Операцията % е приложима само за целочислени аргументи

Примери:

```
int i=1,i1=1,j,k;
j=++i;//j е 2, i е 2
k=i1++;//k е 1, i1 е 2
```

OVERFLOW И UNDERFLOW

За целочислените типове е важно да се има предвид и т.нар. препълване. Това е ситуация при която при аритметична операция се получава резултат (този резултат се присвоява на променлива), който е извън обхвата на съответния тип. За реалните типове термините означават, че стойността е близка до 0 но се различава.

```
#include <iostream>
using namespace std;
int main()
{
    short testVal = 32767; // макс. стойност за short int
    cout << testVal<< endl;
    testVal = testVal + 1; // препълване
    cout << testVal << endl;
    testVal = testVal - 1;
    cout << testVal << endl;
    return 0;
}
```

Резултат

32767
-32768
32767

ПРОСТИ СТАНДАРТНИ ТИПОВЕ В C++ – СТАНДАРТНИ ФУНКЦИИ В MATH.H

Стандартни функции за извършване на някои **математически изчисления** с простите типове са декларирани в заглавния файл `<cmath>` (`math.h`) :

Функция	Действие
abs	Return absolute value of int
acos	Calculate arccosine
asin	Calculate arcsine
atan, atan2	Calculate arctangent
ceil	Find integer ceiling
cos	Calculate cosine
cosh	Calculate hyperbolic cosine
div	Divide one integer by another, returning quotient and remainder
exp	Calculate exponential function
fabs	Find absolute value
floor	Find largest integer less than or equal to argument
fmod	Find floating-point remainder
labs	Return absolute value of long

Функция	Действие
ldiv	Divide one long integer by another, returning quotient and remainder
log	Calculate natural logarithm
log10	Calculate base-10 logarithm
modf	Split argument into integer and fractional parts
pow	Calculate value raised to a power
rand	Get pseudorandom number
sin	Calculate sine
sinh	Calculate hyperbolic sine
sqrt	Find square root
srand	Initialize pseudorandom series
tan	Calculate tangent
tanh	Calculate hyperbolic tangent

ПРОСТИ СТАНДАРТНИ ТИПОВЕ В C++ – СТАНДАРТНИ ФУНКЦИИ В ctype.h

Стандартни функции за обработка на **символния прост тип** са декларирани в заглавния файл `ctype.h`

(1) **is**-функции:

Routine	Character Test Condition
isalnum	Alphanumeric
isalpha	Alphabetic
__isascii	ASCII
isctrl	Control
isdigit	Decimal digit
islower	Lowercase
isprint	Printable
ispunct	Punctuation
isspace	White-space
isupper	Uppercase

(2) **to**-функции:

Routine	Description
tolower	Converts <i>c</i> to lowercase if appropriate
toupper	Converts <i>c</i> to uppercase if appropriate

ДИСКРЕТНИ ТИПОВЕ ДЕФИНИРАНИ ОТ ПРОГРАМИСТА В C++ – ЗНАКОВ МОДЕЛ НА ИЗБРОЕН ТИП

Изброен тип – дискретен тип, чието множеството от стойности е линейно подредено крайно множество, което се задава чрез пряко изброяване на имената на тези стойности

- **Синтаксис** за деклариране на типа:

```
enum <име> незад { <изброяващ списък> незад }
```

Където <изброяващ списък> е:

```
<име> = <константен израз> незад , ...
```

- **Физически модел:** 4 байта
- **Множеството от стойности:** съвкупността от идентификаторите, изброени в скобите при деклариране на типа. На всяка стойност от даден изброен тип се съпоставя целочислена стойност:
 - по подразбиране: 0, 1, 2, ... по реда на изброяването им
 - чрез явно задаване в *изброяващия списък*, като стойностите могат да се повтарят. Ако цялото число, съответно на някоя изброена стойност не е зададена, то е с 1 по-голямо от целочислената стойност на предишното име от списъка
- **Примери:**

```
enum Suit { Diamonds, Hearts, Clubs, Spades } s;  
enum Suit {Diamonds = 5,Hearts,Clubs = 4,Spades=-1};  
enum Color {Red=5, Orange=Red+1, Yellow, Green, Blue};
```

ИЗБРОЕН ТИП В C++ – ОПЕРАЦИИ И РЕЛАЦИИ

- Стойностите от изброен тип се считат еквивалентни на съответните им целочислени стойности. Поради това за тях са валидни всички **операции** валидни и за **целочислените типове**
- Стойностите от изброен тип са подредени според наредбата на съответните им целочислени стойности. Поради това за тях валидни всички **операции за сравнение**
- При **извеждане** (напр. с `cout`) на стойности от изброен тип се извежда техният целочислен еквивалент
- Стойностите от изброен тип не могат пряко да се въвеждат (напр. с `cin`)
- **Примери:**

```
s = sin(s);  
cout<<(Orange<Yellow+1)<<endl;
```

ИЗРАЗИ. ИЗРАЗИ В C++

- Под **израз** (*expression*) в ЕП се разбира правило за изчисляване на някаква стойност от точно определен тип. Изразът се състои от един или повече операнда, свързани помежду си с някои от допустимите операции в езика
- При срещане на израз в една програма се извършва изчисление на израза и се формира една единствена стойност
- При **конструиране на изрази** е необходимо да се съчетават правилно операциите и операндите
- Изразът в езика C++ представлява:
Последователност от операнди и операции, която се използва за една от следните цели:
 - изчисляване на стойност въз основа на операндите (константи, променливи, изрази, израз в кръгли скоби): `-2.16`, `money`, `2*a-3`, `2/(a+3)`, `i++`
 - обозначаване на функции или други обекти: `sqrt(x)`, `myFunc('s',2)`



Изразите, които се използват за последните втората цел ще бъдат разгледани по-подробно по-късно

ИЗЧИСЛЯВАНЕ НА ИЗРАЗИ. ПРИОРИТЕТ НА ОПЕРАЦИИТЕ В C++

- При изчисляване на изразите се прилагат следните правила:
 - в един израз операциите с по-висок приоритет се изчисляват преди тези с по-нисък
 - в случай на няколко последователни операции с един и същ приоритет, изчисленията се извършват отляво надясно
 - изразите със скоби се изчисляват първи
- Операциите изброени по намаляване на техния приоритет са:

Оператор	Действие
()	Function call
()	Conversion
++	Postfix increment
--	Postfix decrement
++	Prefix increment
--	Prefix decrement
*	Dereference
+	Unary plus
-	Arithmetic negation (unary)
!	Logical NOT
*	Multiplication
/	Division
%	Remainder (modulus)
+	Addition
-	Subtraction

Оператор	Действие
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equality
!=	Inequality
&&	Logical AND
	Logical OR
e1?e2:e3	Conditional
=	Assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
+=	Addition assignment
-=	Subtraction assignment

СЪВМЕСТИМОСТ И ПРОВЕРКА НА ТИПОВЕТЕ В ЕП

Съвместимостта (еквивалентност, *type compatibility*) на типове това е свойството им да могат да се заместват като аргументи на изрази, функции, оператори и други

- **Определение:** Ще казваме, че типът T1 е съвместим с типа T2 ако, в случай когато се изисква аргумент от типа T1, на негово място може да се зададе аргумент от типа T2

Проверката на типа (*type checking*)

- **Определение:** Процесът, при който транслаторът проверява дали всички конструкции в програмата имат смисъл по отношение на типовете на константите, променливите, процедурите и другите единици
- **Видове:** варира от строга до либерална в различните ЕП
- **Начини на извършване:** *статична* (по време на компилирането, *static*) и *динамична* (по време на изпълнение, *dynamic*)

ПРЕОБРАЗУВАНЕ НА ТИПОВЕТЕ В ЕП

Преобразуването на типовете (*type conversion*) позволява изрази от действителния тип T_a да се използват в контекста на очакван тип T_e

- **Начини** на преобразуване: **явно** (програмиста пише собствен код за преобразуване на типове); **неявно** (ЕП предоставя специална поддръжка за преобразуване типове)
- **Начини** на задаване на преобразуване в ЕП:
 - **безусловно преобразуване** (неявно, принудително) – преобразуването автоматично се извършват от процесора (ако такова е дефинирано)
 - **специални функции** (напр. `trunc` и `round` са две различни преобразувания от `real` в `integer` в езика Pascal)
 - **претипизиране** (явно, определяне на типа, *type casting*) – чрез указване на типа, към който става преобразуването
- **Видове** преобразувания от семантична гледна:
 - **Преобразуванията без загуби** запазват информацията, съдържаща се в стойността (напр. целочислен в низ; плаваща запетая в двойна точност; знаков в целочислен)
 - **Преобразуванията със загуби** могат да загубят информация (напр. плаваща запетая в целочислен; целочислен в знаков)

ПРЕОБРАЗУВАНЕ НА ТИПОВЕТЕ В C++

Правилата на C++ са доста либерални при задаване на смесени изрази. Реализирани са безусловно преобразуване, специални функции за преобразуване и претипизиране

Преобразуване на типовете се извършва в следните случаи:

- Когато стойност от един тип се присвоява на променлива от друг тип или когато някоя операция преобразува типа на своите операнди преди да бъде извършена
- Когато стойност от един тип явно се преобразува към друг тип
- Когато стойност се предава като аргумент на функция или когато се връща като резултат на функция

ПРЕОБРАЗУВАНЕ НА ТИПОВЕТЕ В C++: БЕЗУСЛОВНО ПРЕОБРАЗУВАНЕ

- **Правила:** В един израз или подизраз всички компоненти се преобразуват безусловно до типа с най-широк диапазон, който присъства в израза по специални правила и зависи от компилатора (вж. примера)

Long double (най-широк)

Double

Float

Unsigned long int

Long int

Unsigned int

Int (най-малък)

- **Пример:**

```
float    fVal;           double  dVal;  int    iVal;
unsigned long ulVal;
dVal = iVal * ulVal; /* iVal се преобразува в unsigned long
                      * Резултатът от умножението се преобразува
                      * до double */
dVal = ulVal + fVal; /* ulVal се преобразува в float
                      * Резултатът от събирането се преобразува
                      * до double */
```



По-безопасно е да се прилага стриктно преобразуване, като се използва някой от другите два начина

ПРЕОБРАЗУВАНЕ НА ТИПОВЕТЕ В C++: ПРЕТИПИЗИРАНЕ

- **Синтаксис:** (<име на тип>) <израз>
- **Семантика:** Резултатът на <израз> се преобразува към типа <име на тип>. Преобразува от кой да е скаларен тип към кой да е друг скаларен тип по правилата, които се използват и при безусловното преобразуване
- **Пример:**

```
float x = 3.1;  
int i;  
i = (int)x; // стойността на i е 3  
x =(float)2;
```
- **Случаи, в които е допустимо претипизирането:**

Тип на резултата	Тип, който се преобразува
Целочислени	Всеки целочислен, с плаваща запетая или тип указател
С плаваща запетая	Всеки аритметичен тип
Тип указател или <code>void *</code>	Всеки целочислен, <code>void *</code> или тип указател
Структура, обединение или масив	—
Тип <code>void</code>	Всеки тип

ПРЕОБРАЗУВАНЕ НА ТИПОВЕТЕ В C++: СПЕЦИАЛНИ ФУНКЦИИ ЗА ПРЕОБРАЗУВАНЕ

- **Синтаксис:** <име на тип> (<израз>)
- **Семантика:** Резултатът на <израз> се преобразува към типа <име на тип>. Преобразува от кой да е скаларен тип към кой да е друг скаларен тип по правилата, които се използват и при безусловното преобразуване

- **Примери:**

```
float f=float(2);
```

```
enum Days
```

```
{    Sunday, Monday, Tuesday, Wednesday,  
    Thursday, Friday, Saturday
```

```
};
```

```
int i;
```

```
Days d = Thursday;
```

```
i = d;    // неявно преобразуване
```

```
cout << "i = " << i << "\n";
```

```
d = 6;    // Грешка !!!
```

```
d = (Days)6;    // Претипизиране
```

```
d = Days( 4 );    // Функция за преобразуване на типа
```



Някои от стандартните функции за обработка на стандартните прости типове, които са декларирани в заглавния файл `math.h` също служат за преобразуване на типовете

CASTING ФУНКЦИИ*

`static_cast< DataType >( Value )`, където `Value` е променлива или литерал(константа), а `DataType` е тип данни до които да се извърши конвертирането

Пример

```
double number = 3.5;

int val;

val = static_cast<int>(number);

cout << val; // извежда се 3
```

* Други , които не се разглеждат

- `dynamic_cast <new_type> (expression)`
- `reinterpret_cast <new_type> (expression)`
- `const_cast <new_type> (expression)`

ЧЕСТО ДОПУСКАНИ ГРЕШКИ ПРИ ИЗПОЛЗВАНЕ НА ПРОСТИТЕ ТИПОВЕ В C++

- В компютъра числата се представят в краен брой битове, което може да доведе до поне два проблема:
 - целочислените стойности са твърде големи, за да бъдат представени
 - реалните стойности са представени приблизително (препоръчително е типът `float` да не се използва)



Във всички тези случаи, ако е възможно компилаторът извежда предупредителни съобщения (*warning*)

- **Пример:**

```
//пример за грешка от закръгляне
double price=3e14;
double price_=price-0.05;
double disc=price-price_;
//трябва де е 0.05
cout<<disc<<"\n";//извежда 0.0625 !!!
```

- Операцията “целочислено деление” може да предизвика проблеми, защото в случай, че и двата аргумента са цели, то в резултата дробната част се игнорира ($10/4$ е 2)